

Camp-Nou: AI-Powered College Timetable Orchestration System

CSD 416 - PROJECT PHASE II

22CSA22 MDL22CS062	Bimal M Binoy
22CSA24 MDL22CS068	Devadath K S
22CSA44 MDL22CS136	Navaneeth A B
22CSA61 MDL22CS182	Sreehari Sujith M R

B. Tech. Computer Science & Engineering



Department of Computer Engineering
Model Engineering College, Ernakulam
Thrikkakara, Kochi 682021
Phone: +91.484.2575370
<http://www.mec.ac.in>
hodcs@mec.ac.in

May 2026

Model Engineering College, Ernakulam
Dept. of Computer Engineering



C E R T I F I C A T E

This is to certify that, this report titled *Camp-Nou: AI-Powered College Timetable
Orchestration System* is a bonafide record of the work done by

22CSA22 MDL22CS062	Bimal M Binoy
22CSA24 MDL22CS068	Devadath K S
22CSA44 MDL22CS136	Navaneeth A B
22CSA61 MDL22CS182	Sreehari Sujit M R

Eighth Semester B. Tech. Computer Science & Engineering
student, for the course work in **CSD 416 - Project Phase II** which is the second part of
the two semester Project Work, under our guidance and supervision, in partial fulfillment of
the requirements for the award of the degree of B. Tech. Computer Science and Engineering
of **APJ Abdul Kalam Technological University**.

Guide

Dr. Manilal D L
Professor
Computer Engineering

Coordinator

Dr. Sindhu L
Professor
Computer Engineering

Head of the Department

Dr. Binu V. P
Professor
Computer Engineering

May 2026

Acknowledgements

We are profoundly grateful to Dr. Manilal D L for his expert guidance and continuous encouragement throughout to see that this project reaches its target from its commencement to its completion. We would like to express deepest appreciation towards Dr. Jayachandran E S, Principal, Govt. Model Engineering college, Kochi, Dr. Binu V P, Head of Department of Computer Engineering and Dr. Sindhu L, Project Coordinator whose invaluable guidance supported us in completing this project. We also express our sincere heartfelt gratitude to all the staff of Computer Engineering Department who helped us directly or indirectly during this course of work. Last but not the least, we thank the Almighty God for guiding us through and enabling us to complete the work within the specified time.

22CSA22 Bimal M Binoy

22CSA24 Devadath K S

22CSA44 Navaneeth A B

22CSA61 Sreehari Sujith M R

Abstract

The generation of academic timetables is a complex, multi-constrained optimization problem that has traditionally been addressed using heuristic algorithms, constraint satisfaction programming, or manual curation. These conventional approaches are time-consuming, prone to scheduling conflicts, and difficult to adapt when resources are scarce or new constraints are introduced. Issues such as room availability, shared laboratory spaces, faculty scheduling overlaps, and dynamic room booking demands significantly complicate the process.

With the rapid advancement of artificial intelligence, particularly Large Language Models (LLMs), there is an opportunity to leverage their reasoning capabilities for intelligent constraint resolution. In this project, we propose Camp-Nou, an AI-powered college timetable orchestration system that integrates LangGraph and LangChain frameworks with a locally hosted language model to autonomously generate, validate, and resolve conflicts in academic timetables.

The system employs a multi-stage pipeline architecture where an LLM-powered agent handles student sectioning and room allocation, while a constraint programming solver (Google OR-Tools CP-SAT) generates the core timetable. A deterministic validation layer ensures all hard constraints are satisfied before persistence. The platform includes a FastAPI backend connected to a Supabase PostgreSQL database, paired with a React-based interactive frontend supporting multi-role access for Administrators, Faculty, and Clubs — enabling end-to-end institutional workflow from bulk data upload and AI-driven scheduling to individual schedule viewing and dynamic club room bookings.

This architecture demonstrates a scalable paradigm for integrating AI-assisted problem-solving within strict deterministic system boundaries, providing an efficient and reliable solution for academic timetable management.

Contents

List of Figures	4
List of Tables	5
1 Introduction	6
1.1 Proposed Project	6
1.1.1 General Background	6
1.1.2 Objectives of project	6
2 System Study Report	8
2.1 Literature Survey	8
2.1.1 Literature Survey Table	11
3 Problem Statement and Proposed Solution	14
3.0.1 Problem Statement	14
3.0.2 Proposed Solution	14
4 Software Requirement Specification	16
4.1 Introduction	16
4.1.1 Purpose	16
4.1.2 Intended Audience and Reading Suggestions	16
4.1.3 Project Scope	16
4.1.4 Overview of Developer's Responsibilities	16
4.2 Overall Description	17
4.2.1 Product Perspective	17
4.2.2 Product Functions	17
4.2.3 User Classes and Characteristics	18
4.2.4 Operating Environment	18
4.2.5 Design and Implementation Constraints	18
4.2.6 General Constraints	18
4.2.7 Assumptions and Dependencies	19
4.3 External Interface Requirements	19
4.3.1 User Interfaces	19
4.3.2 Hardware Interfaces	19
4.3.3 Software Interfaces	19
4.3.4 Communication Interfaces	19
4.4 Hardware and Software Requirements	20

4.4.1	Hardware Requirements	20
4.4.2	Software Requirements	20
4.5	Functional Requirements	20
4.5.1	Data Upload and Management	20
4.5.2	Student Sectioning	21
4.5.3	Timetable Generation	21
4.5.4	Room Booking	21
4.6	Non Functional Requirements	22
4.6.1	Performance Requirements	22
4.6.2	Safety Requirements	22
4.6.3	Software Quality Attributes	22
5	Dataset and Data Modeling	23
5.1	Introduction	23
5.2	Data Sources	23
5.3	Database Schema	23
5.4	Data Preprocessing	24
5.5	Entity Relationships	24
5.6	Summary	25
6	System Design	26
6.1	Use Case Diagram	26
6.2	System Architecture	27
6.2.1	Feature Extraction and Constraint Building	28
6.2.2	Timetable Generation	28
6.2.3	Validation	28
6.3	Input Design	28
6.4	Database Design	29
6.5	Libraries and Packages Used	29
6.6	Module Description	30
6.6.1	Data Upload	30
6.6.2	Student Sectioning	30
6.6.3	Timetable Generation	30
6.6.4	Authentication	30
6.6.5	Room Booking	30
6.6.6	Frontend Interface	31
7	Data Flow Diagram	32
7.1	Level 0 DFD	32
7.2	Level 1 DFD	33
7.3	Level 2 DFD	33
7.3.1	Data Upload and Parsing	34
7.3.2	Timetable Generation	35
7.3.3	Room Booking	36
8	Implementation	37

8.1	Algorithms Used	37
8.1.1	Student Sectioning Algorithm	37
8.1.2	Timetable Generation using CP-SAT	37
8.1.3	Authentication and Authorization	38
8.2	Development Tools	38
8.2.1	Python Backend Stack	38
8.2.2	AI and Constraint Programming	39
8.2.3	Frontend Development	39
9	Unit Testing	40
10	Graphical User Interface	50
10.1	GUI Overview	50
10.1.1	Description of the User Interface	50
10.1.2	Objectives and Actions	50
10.1.3	Interface Design Rules	50
10.2	Main GUI Components	51
10.2.1	Components	51
10.3	Screen Images	52
10.3.1	Login Interface	52
10.3.2	Admin Dashboard	53
10.3.3	Data Upload Module	54
10.3.4	Timetable Grid View	55
10.3.5	Faculty Schedule View	56
10.3.6	Room Booking Interface	57
10.3.7	Booking Management	58
11	Results	59
11.1	Performance Evaluation	59
11.1.1	Scheduling Correctness	59
11.1.2	Generation Speed	59
11.1.3	AI Reliability	59
11.2	Multi-Role Access Results	59
11.3	Room Booking Workflow	60
11.4	Limitations	60
11.5	Summary	60
12	Conclusion	61
13	Future Scope	62
	Bibliography	63

List of Figures

6.1	Use Case Diagram	26
6.2	Architecture Diagram	27
6.3	Database Design	29
7.1	DFD Level 0	32
7.2	DFD Level 1	33
7.3	DFD Level 2	34
7.4	DFD Level 2	35
7.5	DFD Level 2	36
10.1	Login Interface	52
10.2	Admin Interface	53
10.3	Data Upload Module	54
10.4	Timetable Grid View	55
10.5	Faculty Schedule View	56
10.6	Room Booking Interface	57
10.7	Booking Management	58

List of Tables

2.1	Comprehensive Comparison of Literature Survey Papers for Timetable Scheduling	13
9.1	Data Upload Module Testing	41
9.2	Timetable Generation Testing	44
9.3	Authentication and Authorization Testing	47
9.4	Room Booking Module Testing	49

Chapter 1

Introduction

1.1 Proposed Project

1.1.1 General Background

Academic timetabling is one of the most persistent and computationally demanding administrative challenges in educational institutions. The objective is to assign a finite set of events — lectures, laboratory sessions, and tutorials — to a limited number of resources including classrooms, faculty members, lab spaces, and time slots, over a defined period while strictly adhering to numerous hard and soft constraints.

Hard constraints include preventing faculty from teaching two classes simultaneously, ensuring no room hosts multiple distinct sessions concurrently, and guaranteeing that laboratory sessions occupy consecutive time slots. Soft constraints involve optimizing the distribution of sessions across a week, avoiding back-to-back lectures of the same subject, and preferring lab sessions earlier in the week.

While deterministic methods such as genetic algorithms and simulated annealing have improved upon manual scheduling, they are rigid and require significant programmatic retuning whenever the foundational rule set changes. The advent of highly capable Large Language Models (LLMs) has introduced the potential to model this constraint satisfaction problem as a natural language reasoning task, complemented by formal constraint solvers for guaranteed correctness.

Furthermore, timetable generation systems in educational institutions are often isolated from day-to-day management realities such as club room bookings, faculty schedule accessibility, and rapid scheduling modifications. A unified, intelligent system capable of autonomous scheduling, constraint resolution, and daily management is required to eliminate the administrative bottleneck of college timetable planning.

1.1.2 Objectives of project

Objectives

- Develop an AI-powered timetable generation system that leverages LLMs for intelligent student sectioning and room allocation.

- Implement constraint programming using Google OR-Tools CP-SAT solver for generating conflict-free timetables.
- Utilize LangGraph to structure the scheduling pipeline into distinct states: data fetching, constraint building, solving, validation, and persistence.
- Build a robust FastAPI backend with Supabase PostgreSQL integration to manage institutional entities efficiently.
- Design a unified multi-role React frontend enabling Administrators to upload data and manage bookings, Faculty to access individualized timetables, and Clubs to book available rooms dynamically.
- Ensure system reliability through deterministic fallback mechanisms when AI components produce invalid outputs.

Chapter 2

System Study Report

2.1 Literature Survey

1. Automated University Timetabling Using Constraint Logic Programming (2018)

This fundamental paper explores utilizing Prolog and Constraint Logic Programming (CLP) for university timetabling. The methodology represents faculty, rooms, and subjects as logical constraints, evaluating valid domain states. While the solution effectively guarantees conflict-free schedules through exhaustive state-space exploration, it scales poorly. As the number of students, rooms, and constraints grows, the state-space explosion forces execution times into hours or days, making it impractical for real-world institutions with hundreds of sections and limited computational resources. The approach also lacks the ability to dynamically resolve impossible schedules without manual relaxation of constraints.

2. Genetic Algorithms for University Course Timetabling (2020)

This study focuses on evolutionary algorithms to solve scheduling problems. By encoding timetables as chromosomes and using crossover and mutation operators, the system progressively minimizes constraint violations using a fitness function. While computationally faster than CLP for finding satisfactory schedules, GA-based systems struggle with explainability — it is difficult to understand why specific scheduling decisions were made. Moreover, adding a complex rule such as using a lab as a temporary classroom requires a complete mathematical rewrite of the fitness function, making the approach inflexible to institutional requirement changes.

3. LLMs for Combinatorial Optimization and Constraint Satisfaction (2023)

Recent advancements detailed in this paper demonstrate the efficacy of utilizing modern Large Language Models for combinatorial optimization tasks. The paper outlines methods of translating mathematical constraints into natural language prompts, enabling the model to reason through state transitions. The key advantage is adaptability: LLMs can intuitively parse new rules and propose logical resolutions without requiring algorithmic rewrites. However, the paper notes that LLMs occasionally hallucinate solutions that violate constraints, necessitating deterministic post-validation layers to ensure correctness.

4. Agentic Workflows in Educational Administration using LangChain (2024)

This work highlights using framework architectures like LangChain to build multi-step agentic workflows for administrative tasks. By compartmentalizing complex operations into distinct nodes — fetching data, proposing a solution via AI, validating via deterministic code, and iterating — the system achieves high accuracy and reliability. This aligns directly with our proposed architecture, as it neutralizes the LLM hallucination risk by enforcing a code-based safety net over the generated output.

5. CP-SAT Solver for Scheduling Problems (2022)

This paper presents the application of Google’s OR-Tools CP-SAT solver for scheduling and resource allocation problems. The CP-SAT solver uses constraint propagation and SAT-based search techniques to efficiently explore the solution space. The solver guarantees optimality or feasibility within defined time limits and supports complex constraints such as interval variables for consecutive slot assignments. The approach is highly deterministic and reproducible, making it suitable for mission-critical scheduling applications where consistency is paramount.

6. Multi-Role Web Applications for Educational Institution Management (2023)

This study focuses on the design and implementation of role-based access control systems in educational web applications. The paper discusses the challenges of managing multiple user types — administrators, faculty, and student organizations — within a unified platform. Authentication and authorization patterns using JWT-based session management are explored, along with event-driven architectures for real-time notifications. The approach emphasizes the importance of separating concerns between data management, scheduling logic, and user interface components.

7. FastAPI and SQLAlchemy for High-Performance Backend Services (2023)

This paper evaluates the performance characteristics of FastAPI combined with SQLAlchemy ORM for building scalable backend services. The study demonstrates that FastAPI’s asynchronous request handling, combined with SQLAlchemy’s flexible ORM layer, provides excellent throughput for CRUD-heavy applications. Connection pooling strategies and transaction management patterns are discussed, along with best practices for integrating with cloud-hosted PostgreSQL databases such as Supabase.

8. Room Booking and Resource Management Systems for Universities (2024)

This paper presents a comprehensive room booking system integrated with institutional scheduling. The system allows student organizations to request room reservations, which are then validated against existing timetable schedules to prevent conflicts. Administrator approval workflows and notification systems are implemented to streamline the booking process. The paper highlights the importance of maintaining a single source of truth for room availability across both scheduled classes and ad-hoc bookings.

9. React-Based Single Page Applications for Dashboard Interfaces (2024)

This study explores the design patterns for building responsive, role-based dashboard interfaces using React and Vite. Component composition strategies, state management approaches, and responsive layout techniques are discussed. The paper emphasizes the importance of providing role-specific views that surface relevant information without overwhelming users with unnecessary complexity.

10. Hybrid AI and Constraint Programming Approaches for Resource Allocation (2025)

This work presents a hybrid approach that combines LLM-based reasoning with formal constraint solvers for resource allocation problems. The LLM handles high-level strategic decisions such as grouping and prioritization, while the constraint solver ensures that all hard constraints are satisfied. This two-stage approach leverages the strengths of both paradigms — the flexibility and adaptability of AI with the mathematical rigor of constraint programming — resulting in solutions that are both creative and provably correct.

2.1.1 Literature Survey Table

Year	Paper Title & Authors	Methodology	Advantages	Disadvantages
2018	Automated University Timetabling Using Constraint Logic Programming	Uses Prolog-based CLP to model faculty, rooms, and subjects as logical constraints for exhaustive state-space exploration.	Guarantees conflict-free schedules. Mathematically rigorous. Complete search over solution space.	Very low scalability. Fails on large datasets. Cannot dynamically resolve impossible schedules.
2020	Genetic Algorithms for University Course Timetabling	Encodes timetables as chromosomes using crossover and mutation. Uses fitness function to minimize constraint violations.	Explores massive search spaces quickly. Can find good-enough solutions fast. Handles soft constraints.	Codebase is rigid. Adding rules requires rewriting fitness function. Poor explainability of decisions.
2023	LLMs for Combinatorial Optimization and Constraint Satisfaction	Translates mathematical constraints into natural language prompts for LLM reasoning and solution generation.	Highly adaptive and logical. No algorithmic rewrites needed for new rules. Intuitive constraint parsing.	Prone to hallucinations. May violate constraints. Requires deterministic validation layer.
2024	Agentic Workflows in Educational Administration using LangChain	Multi-node graph orchestration of LLMs with deterministic code-based validation between reasoning steps.	High reliability through self-correcting operations. Separates AI reasoning from validation.	Dependent on latency. System prompt complexity affects output quality.

Year	Paper Title & Authors	Methodology	Advantages	Disadvantages
2022	CP-SAT Solver for Scheduling Problems	Uses Google OR-Tools constraint propagation and SAT-based search for scheduling and resource allocation.	Guaranteed optimality within time limits. Deterministic and reproducible. Supports interval constraints.	Requires careful constraint modeling. May report infeasible for over-constrained problems.
2023	Multi-Role Web Applications for Educational Institution Management	Role-based access control with JWT authentication for multi-user educational platforms.	Clean separation of user responsibilities. Secure authentication patterns. Real-time notifications.	Complex authorization logic. Increased testing requirements for role combinations.
2023	FastAPI and SQLAlchemy for High-Performance Backend Services	Evaluates async FastAPI with SQLAlchemy ORM for CRUD-heavy applications with cloud PostgreSQL.	Excellent throughput. Flexible ORM mapping. Connection pooling support. Easy cloud DB integration.	Async patterns add complexity. ORM overhead for complex queries.
2024	Room Booking and Resource Management Systems for Universities	Integrated room booking with timetable validation. Admin approval workflows with notification systems.	Single source of truth for room availability. Prevents scheduling conflicts. Streamlined workflows.	Requires tight integration with scheduling system. Complex state management for bookings.

Year	Paper Title & Authors	Methodology	Advantages	Disadvantages
2024	React-Based Single Page Applications for Dashboard Interfaces	Component composition and role-specific view rendering using React and Vite build system.	Responsive design. Role-specific dashboards. Fast development with hot module replacement.	Client-side complexity. State management challenges. Bundle size considerations.
2025	Hybrid AI and Constraint Programming for Resource Allocation	Combines LLM reasoning for strategic decisions with formal constraint solvers for hard constraint satisfaction.	Leverages strengths of both AI and CP. Creative yet provably correct solutions. Highly adaptable.	Two-stage complexity. Requires careful interface design between AI and solver.

Table 2.1: Comprehensive Comparison of Literature Survey Papers for Timetable Scheduling

Chapter 3

Problem Statement and Proposed Solution

3.0.1 Problem Statement

Generating a scalable, conflict-free college timetable is a resource-intensive challenge due to several inherent complexities. Academic scheduling involves assigning multiple classes to limited rooms and faculty across fixed time slots while satisfying numerous constraints simultaneously. Hard constraints — such as preventing faculty double-booking and ensuring lab sessions occupy consecutive slots — must be strictly enforced, while soft constraints — such as distributing subjects evenly across the week — should be optimized where possible.

Another major challenge is resource scarcity. When the number of classes exceeds the number of available classrooms, the system must intelligently stagger sessions, exploit lab-period vacancies (when students leave classrooms for labs), and potentially repurpose unused lab rooms as temporary lecture halls. Conventional deterministic algorithms either fail to resolve such complex scenarios or require exponential computation time.

Furthermore, existing timetable generation systems are often isolated from day-to-day college management realities. They do not integrate with processes such as dynamic room availability tracking, club event scheduling, or faculty accessibility. This fragmentation forces administrators to maintain multiple disconnected systems, increasing the risk of scheduling conflicts.

Additionally, traditional approaches lack transparency when unable to find a valid schedule — they simply crash or produce incomplete results without explaining which constraints are unsatisfiable. This makes it difficult for administrators to understand and resolve the underlying issues.

3.0.2 Proposed Solution

The proposed system is Camp-Nou, an AI-powered college timetable orchestration system that combines intelligent AI reasoning with formal constraint programming to automatically generate, validate, and manage academic timetables.

The system employs a multi-stage pipeline architecture implemented using LangGraph:

1. **Student Sectioning and Room Allocation:** An LLM-powered agent intelligently segments students into branch-based sections (e.g., CS-A, CS-B, EC-A) based on room capacities, and assigns appropriate classrooms to each section. Deterministic fallback logic ensures valid assignments even if the AI produces incorrect output.
2. **Timetable Generation:** The Google OR-Tools CP-SAT constraint programming solver generates the core timetable by modeling all hard constraints (faculty availability, room occupancy, lab consecutiveness) as boolean satisfiability conditions. The solver guarantees mathematical correctness within defined time limits.
3. **Validation and Conflict Resolution:** A deterministic validation layer checks the generated timetable for double-bookings and constraint violations. Any issues are logged as warnings for administrator review.
4. **Persistence:** Valid timetables are persisted to a Supabase PostgreSQL database, serving as the single source of truth for the entire institution.

To complement the intelligent backend, the solution incorporates a full-stack, multi-role infrastructure. Administrators supervise data inputs through Excel file uploads and trigger schedule generation. Faculty members access personalized timetable dashboards with PDF download capabilities. Clubs can securely book available rooms through a request-approval workflow, relying on the same room availability data used by the scheduling system.

Overall, the proposed solution provides an efficient, scalable, and transparent approach to academic timetable management, reducing reliance on manual expertise and enabling automated analysis of complex scheduling scenarios.

Chapter 4

Software Requirement Specification

4.1 Introduction

4.1.1 Purpose

The purpose of this document is to describe the functional and non-functional requirements of the Camp-Nou AI-Powered College Timetable Orchestration System. This document provides a comprehensive description of the features, behavior, and constraints of the system. It outlines how the system processes institutional data, generates timetables using AI and constraint programming, and manages multi-role access for administrators, faculty, and clubs.

4.1.2 Intended Audience and Reading Suggestions

The intended audience for this system includes college administrators responsible for scheduling, faculty members who need to access their teaching schedules, and student club representatives who require room bookings. Developers and researchers can refer to this document for implementation details and system design patterns.

4.1.3 Project Scope

The system aims to develop an intelligent solution that can generate academic timetables from institutional data using AI and constraint programming techniques. The system accepts data input through Excel file uploads, processes it through an AI-powered pipeline, and generates conflict-free timetables. In addition, the system provides multi-role access, room booking management, and timetable visualization capabilities.

4.1.4 Overview of Developer's Responsibilities

- Develop a robust system for conflict-free timetable generation using constraint programming.
- Design AI-powered student sectioning and room allocation mechanisms with deterministic fallbacks.

- Implement multi-role authentication and authorization for administrators, faculty, and clubs.
- Ensure user-friendly interface for data upload, timetable viewing, and room booking management.
- Optimize the system for performance and scalability across institutional deployments.

4.2 Overall Description

4.2.1 Product Perspective

The system is designed as a full-stack web application consisting of a React frontend, FastAPI backend, and Supabase PostgreSQL database. It integrates AI reasoning through LangChain/LangGraph and constraint programming through Google OR-Tools CP-SAT. The system can be deployed as a cloud-hosted or self-hosted application.

4.2.2 Product Functions

The system performs the following functions:

Data Management

- Accept institutional data through Excel file uploads (rooms, students, subjects, faculty).
- Parse and validate uploaded data before database persistence.
- Normalize data into relational database tables with proper foreign key relationships.

Timetable Generation

- Automatically section students into branch-based groups using AI.
- Allocate classrooms to sections based on capacity constraints.
- Generate conflict-free timetables using CP-SAT constraint solver.
- Validate generated timetables for hard constraint satisfaction.

Multi-Role Access

- Administrator dashboard for data management and schedule generation.
- Faculty view for personalized timetable access and PDF download.
- Club interface for room booking requests with admin approval workflow.

Room Booking

- Allow clubs to request room bookings for events.
- Enable administrators to approve or reject booking requests.
- Maintain room availability as single source of truth.

4.2.3 User Classes and Characteristics

1. **Administrator** – Responsible for uploading institutional data, generating timetables, managing room bookings, and overseeing system operations. Requires full access to all system features.
2. **Faculty** – Users who need to view their assigned teaching schedules and download timetables as PDF documents. They require read-only access to timetable data filtered by their identity.
3. **Club** – Student organization representatives who can request room bookings for events. They require access to the booking interface and can view the status of their requests.

4.2.4 Operating Environment

- Frontend runs on any modern web browser supporting JavaScript ES6+.
- Backend requires Python 3.10+ environment with FastAPI framework.
- Database hosted on Supabase (PostgreSQL) with SSL connection support.
- AI inference requires OpenAI-compatible API endpoint (local or cloud).
- Development server runs via Uvicorn (backend) and Vite (frontend).

4.2.5 Design and Implementation Constraints

- Timetable generation quality depends on quality and completeness of uploaded data.
- CP-SAT solver has a configurable time limit (default 60 seconds) for finding solutions.
- AI-powered nodes must have deterministic fallback mechanisms.
- System must support standard Excel file formats for data upload.
- Database schema must support nullable foreign keys for suspended timetable entries.

4.2.6 General Constraints

1. **Accuracy:** Generated timetables must be free of hard constraint violations.
2. **Performance:** System should generate timetables within reasonable time (under 2 minutes).
3. **Scalability:** Should support multiple branches, sections, and room configurations.
4. **Usability:** Interface must be intuitive for non-technical administrators.

4.2.7 Assumptions and Dependencies

- Uploaded Excel files follow the expected column structure and naming conventions.
- The database connection (Supabase) is available and has sufficient capacity.
- For AI-powered features, an OpenAI-compatible API endpoint is accessible.
- Users have basic knowledge of web application usage and Excel file preparation.
- System depends on libraries such as LangChain, LangGraph, OR-Tools, SQLAlchemy, FastAPI, and React.

4.3 External Interface Requirements

4.3.1 User Interfaces

The system provides a role-based web interface where administrators can upload Excel files, generate timetables, and manage bookings. Faculty members can view their personalized schedules and download PDFs. Club representatives can submit and track room booking requests.

4.3.2 Hardware Interfaces

- Computer or mobile device with modern web browser for accessing the application.
- Server hardware capable of running Python backend and constraint solver computations.

4.3.3 Software Interfaces

- Python libraries: FastAPI, SQLAlchemy, LangChain, LangGraph, OR-Tools, Pandas.
- Frontend: React with Vite build system.
- Database: Supabase PostgreSQL with psycopg2-binary driver.
- AI: OpenAI-compatible API via langchain-openai package.

4.3.4 Communication Interfaces

- HTTP/REST APIs for communication between React frontend and FastAPI backend.
- PostgreSQL wire protocol for database communication via SQLAlchemy ORM.
- HTTPS for secure communication with cloud services.

4.4 Hardware and Software Requirements

4.4.1 Hardware Requirements

1. A system with at least an Intel i5 processor or equivalent is required to ensure smooth execution of the constraint solver and backend processing.
2. A minimum of 8 GB RAM is required for efficient handling of constraint programming computations and concurrent API requests. Higher RAM is recommended for larger institutional datasets.
3. At least 5 GB of free disk space is required to store the application, database configurations, and generated timetable data.
4. Network connectivity is required for communication with the Supabase cloud database and AI inference endpoints.
5. A GPU is not required as the primary computation uses CPU-based constraint programming rather than neural network inference.

4.4.2 Software Requirements

1. Python 3.10+ Programming Environment. The backend is implemented using Python, providing access to constraint programming, ORM, and web framework libraries.
2. FastAPI Web Framework. Used for building high-performance REST APIs with automatic OpenAPI documentation generation and request validation.
3. SQLAlchemy ORM. Provides Object-Relational Mapping for database interactions, supporting PostgreSQL through psycopg2-binary driver.
4. Google OR-Tools. Provides the CP-SAT constraint programming solver used for mathematically rigorous timetable generation.
5. LangChain and LangGraph. Used for building AI-powered agentic workflows with structured state management and node-based execution graphs.
6. React with Vite. The frontend is built using React for component-based UI development and Vite for fast build and hot module replacement.
7. Supabase PostgreSQL. Cloud-hosted PostgreSQL database providing high availability, SSL connections, and web-based administration.

4.5 Functional Requirements

4.5.1 Data Upload and Management

Major functional requirements for the Data Upload module are:-

1. Excel Upload: Accept Excel files containing room, student, subject, and faculty data.
2. Data Parsing: Parse uploaded Excel files using Pandas and validate data integrity.
3. Database Persistence: Store parsed data in normalized relational tables with appropriate foreign keys.
4. Data Validation: Ensure uploaded data meets required format and completeness standards.

4.5.2 Student Sectioning

Major functional requirements for the Student Sectioning module are:-

1. AI-Powered Grouping: Use LLM to intelligently group students into branch-based sections.
2. Capacity Constraint: Ensure each section size does not exceed the assigned room capacity.
3. Deterministic Fallback: Provide sorted-by-ID fallback if AI produces invalid groupings.
4. Section Naming: Generate descriptive section names with branch prefixes (e.g., CS-A, EC-B).

4.5.3 Timetable Generation

Major functional requirements for the Timetable Generation module are:-

1. Constraint Modeling: Model all scheduling constraints as CP-SAT boolean satisfiability conditions.
2. Solver Execution: Execute OR-Tools CP-SAT solver with configurable time limits.
3. Lab Handling: Ensure lab sessions occupy exactly 3 consecutive slots on the same day.
4. Result Extraction: Extract solved variable assignments into structured timetable format.

4.5.4 Room Booking

Major functional requirements for the Room Booking module are:-

1. Booking Request: Allow clubs to submit room booking requests with event details.
2. Admin Approval: Enable administrators to approve or reject booking requests with optional notes.
3. Availability Check: Validate room availability against existing timetable and bookings.
4. Status Tracking: Track booking status through pending, approved, and rejected states.

4.6 Non Functional Requirements

4.6.1 Performance Requirements

Major performance requirements are:

1. The system should generate complete timetables within 120 seconds for typical institutional datasets.
2. The constraint solver should find feasible solutions within the configured time limit (60 seconds default).
3. API endpoints should respond within 500ms for standard CRUD operations.
4. The frontend should render timetable grids smoothly without blocking the main thread.

4.6.2 Safety Requirements

The system should ensure safe and reliable operation under different conditions. The system must handle invalid or incomplete Excel uploads gracefully without crashing. Users should be provided with clear error messages and warnings when data quality issues are detected.

Additionally, the system should clearly communicate when constraint solver cannot find a feasible solution, providing explanatory warnings rather than producing incomplete timetables silently. Password-based authentication with salt-hashed credentials ensures secure multi-role access.

4.6.3 Software Quality Attributes

The system consists of the following quality attributes:

- **Reliability** - The system should consistently generate valid, conflict-free timetables with deterministic fallback mechanisms.
- **Availability** - The system should be accessible to users through web browsers with minimal downtime.
- **Usability** - The role-based interface should present only relevant features to each user type.
- **Scalability** - The system should support increasing numbers of sections, rooms, and subjects.
- **Maintainability** - The modular architecture should allow easy updates to individual components.

Chapter 5

Dataset and Data Modeling

5.1 Introduction

The Camp-Nou system operates on structured institutional data that represents the physical and academic resources of a college. Unlike machine learning systems that require large training datasets, our system uses operational data — rooms, students, subjects, faculty, and their relationships — as input to the scheduling pipeline. The quality and completeness of this data directly determines the quality of the generated timetable.

5.2 Data Sources

The system accepts input data through three standardized Excel files uploaded by the administrator:

1. **Rooms Data (rooms.xlsx):** Contains information about all physical spaces including classrooms and laboratories. Each room entry specifies the room name, seating capacity, and room type (classroom or lab).
2. **Students Data (students.xlsx):** Contains the list of all students with their names and branch assignments. This data is used by the AI-powered student sectioning module to create balanced class sections.
3. **Subjects and Faculty Data (subjects_faculty.xlsx):** Contains subject definitions (name, lab/lecture classification, hours per week) and faculty-subject-class mappings that specify which faculty teaches which subject to which class section.

5.3 Database Schema

The system uses a normalized relational database hosted on Supabase PostgreSQL. The schema consists of the following core tables:

- **Rooms:** Stores room information with fields for id, name, capacity, room_type (classroom/lab), and created_at timestamp.

- **Students:** Stores student records with fields for id, name, branch, and created_at timestamp.
- **Classrooms:** Stores the result of student sectioning with fields for id, room_id, class_id, students (array of student names), and created_at timestamp.
- **Subjects:** Stores academic subjects with fields for id, name, is_lab (boolean), hours_per_week, and created_at timestamp.
- **Faculty:** Stores faculty members with fields for id, name, and created_at timestamp.
- **SubjectFacultyMapping:** Maps subjects to faculty and class sections with fields for id, subject_id, faculty_id, class_name, and created_at timestamp.
- **Timetable:** Stores the generated timetable entries with fields for id, class_name, day, slot (1-6), subject_id, faculty_id, room_id, is_lab_period, status (scheduled/suspended), and created_at timestamp.
- **AppUser:** Stores authenticated users with fields for id, name, email, role (administrator/faculty/club), club_name, password_hash, password_salt, and created_at timestamp.
- **RoomBooking:** Stores room booking requests with fields for id, event_name, club_name, requested_by_user_id, room_id, room_name, event_date, start_time, end_time, status (pending/approved/rejected), admin_note, created_at, and updated_at timestamps.

5.4 Data Preprocessing

Before feeding data into the scheduling pipeline, several preprocessing steps are performed:

1. **Excel Parsing:** Uploaded Excel files are parsed using Pandas with column validation and type checking.
2. **Deduplication:** Duplicate entries are identified and resolved during database insertion.
3. **Foreign Key Resolution:** Subject-faculty mappings are resolved against existing subject and faculty records.
4. **Room Classification:** Rooms are automatically classified as classrooms or labs based on the room_type field.
5. **Default Seeding:** Default room configurations are seeded on application startup to ensure baseline data availability.

5.5 Entity Relationships

The database follows a relational model with the following key relationships:

- SubjectFacultyMapping references both Subjects and Faculty through foreign keys.

- Timetable entries reference Subjects, Faculty, and Rooms through nullable foreign keys (nullable for suspended entries).
- RoomBooking references Rooms and AppUser through foreign keys.
- Classrooms reference Rooms through room_id for section-room assignment tracking.

5.6 Summary

The data modeling approach prioritizes normalization, referential integrity, and extensibility. By storing all institutional data in a well-structured PostgreSQL database, the system ensures that the AI and constraint programming components receive accurate, consistent input for timetable generation. The multi-table design also supports the room booking workflow and multi-role access control without schema modifications.

Chapter 6

System Design

6.1 Use Case Diagram

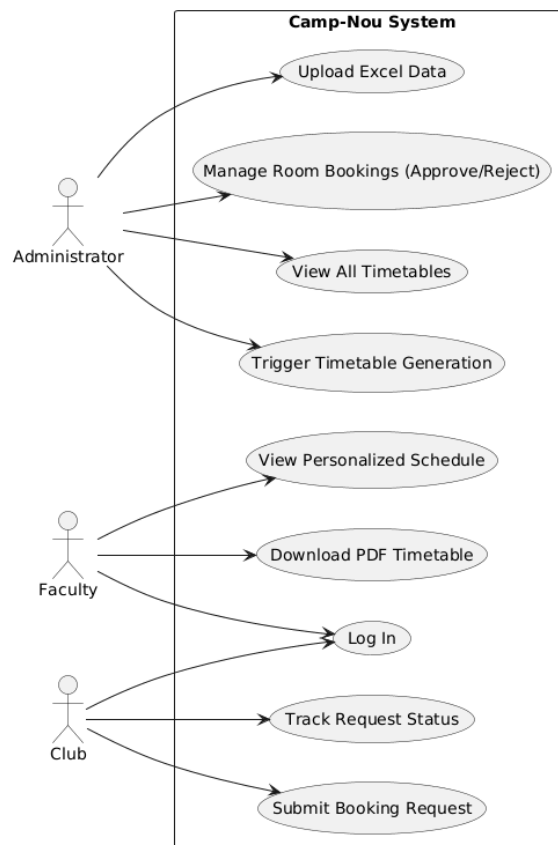


Figure 6.1: Use Case Diagram

The three main actors of the system are the Administrator, Faculty, and Club. The administrator interacts with the system by uploading Excel data files containing room, student, subject, and

faculty information. The administrator can trigger timetable generation, view all generated timetables, and manage room booking requests by approving or rejecting them. Faculty members can log in to view their personalized timetable schedules and download them as PDF documents. Club representatives can log in to submit room booking requests for events and track the status of their requests.

6.2 System Architecture

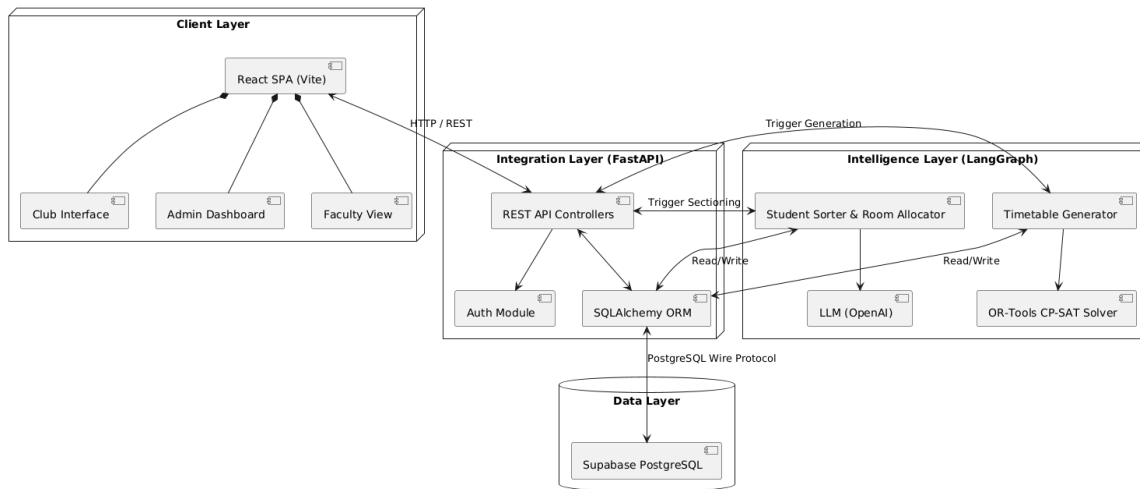


Figure 6.2: Architecture Diagram

Description

The Camp-Nou system follows a decoupled, layered architecture consisting of four primary layers: the Client Layer, Integration Layer, Data Layer, and Intelligence Layer.

The **Client Layer** is a React single-page application built with Vite that provides role-specific dashboards. It communicates with the backend exclusively through REST API calls, maintaining separation of concerns between presentation and business logic.

The **Integration Layer** is built with FastAPI and handles HTTP routing, request validation, JSON marshaling, and database communication via SQLAlchemy ORM. It exposes RESTful endpoints organized into six modules: authentication, rooms, students, subjects/faculty, timetable, and bookings.

The **Data Layer** is hosted on Supabase, providing a high-availability PostgreSQL database with SSL connections. All institutional data, generated timetables, user accounts, and booking records are persisted here as the single source of truth.

The **Intelligence Layer** consists of two LangGraph state machines: the Student Sorter and Room Allocator graph (powered by an LLM via langchain-openai), and the Timetable Generator graph (powered by Google OR-Tools CP-SAT solver). These operate as stateful workflows with deterministic validation checkpoints.

6.2.1 Feature Extraction and Constraint Building

The constraint building module transforms raw institutional data into a structured constraint matrix suitable for the CP-SAT solver. For each class section, it computes the teaching load by cross-referencing subject-faculty mappings with subject definitions, determining the total hours per week for each subject and whether it requires a lab room type. The constraint matrix is stored as metadata within the timetable state for use by subsequent processing nodes.

6.2.2 Timetable Generation

The timetable generation module uses the Google OR-Tools CP-SAT solver to create a conflict-free schedule. It models each teaching session as a set of integer variables (day, slot, room) and defines hard constraints as boolean satisfiability conditions: no faculty double-booking across classes, no room double-booking across sessions, and lab sessions occupying exactly three consecutive slots. The solver explores the solution space using constraint propagation and SAT-based search, returning an optimal or feasible assignment within the configured time limit.

6.2.3 Validation

The validation module performs post-generation verification of the timetable. It builds faculty and room occupancy maps from the generated timetable entries and checks for double-bookings. Any violations are logged as warnings and reported to the administrator. The validation node supports conditional branching — if violations are found, the graph can retry the solve step (currently configured to proceed with warnings).

6.3 Input Design

The system consists of three types of inputs:

- **Excel Data Files:** The system accepts three Excel files containing room data, student data, and subject-faculty mapping data. Each file must follow the expected column naming conventions.
- **User Authentication:** Users authenticate by providing email and password credentials, which are validated against salt-hashed entries in the database.
- **Booking Requests:** Club users submit booking requests specifying event name, room, date, and time range through the web interface.

6.4 Database Design

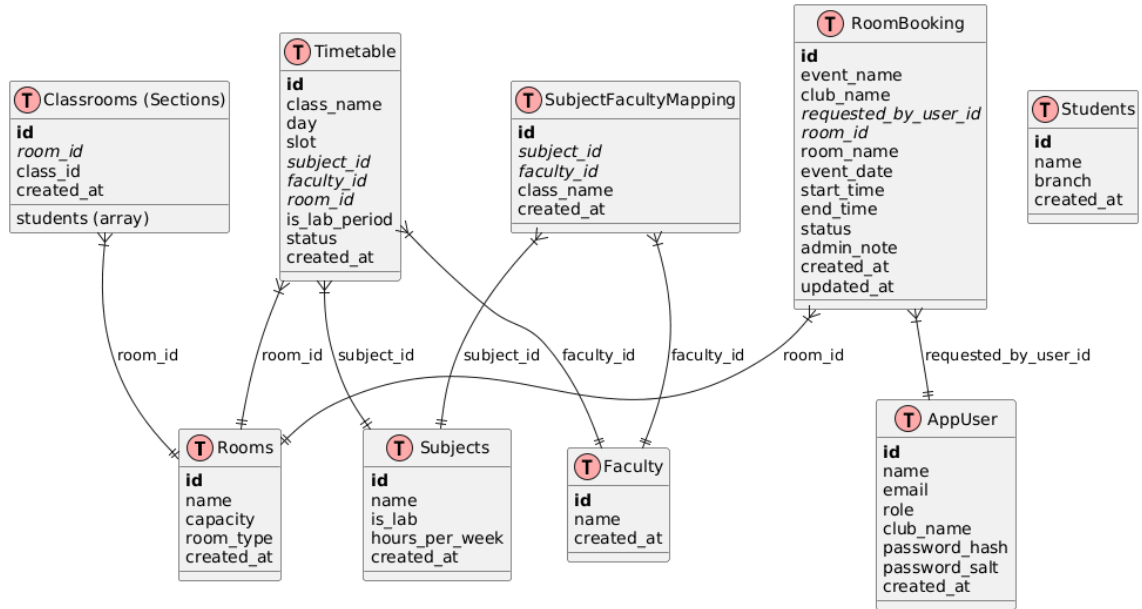


Figure 6.3: Database Design

The database includes information related to institutional entities, scheduling results, user accounts, and booking records.

- **Institutional Data:** Rooms, Students, Subjects, Faculty, and SubjectFacultyMapping tables store the raw institutional configuration.
- **Scheduling Data:** Classrooms table stores section-room assignments, and Timetable table stores the generated schedule entries with slot-level granularity.
- **User Data:** AppUser table stores authenticated user accounts with role-based access control.
- **Booking Data:** RoomBooking table stores room reservation requests with approval workflow status tracking.

6.5 Libraries and Packages Used

- **FastAPI:** High-performance async web framework for building REST APIs with automatic documentation.
- **SQLAlchemy:** Object-Relational Mapping library for database interactions with PostgreSQL.
- **LangChain / LangGraph:** Framework for building AI-powered agentic workflows with state graph orchestration.

- **Google OR-Tools:** Constraint programming library providing the CP-SAT solver for timetable optimization.
- **Pandas:** Data manipulation library used for parsing and processing uploaded Excel files.
- **React:** Component-based JavaScript library for building the interactive frontend interface.
- **Vite:** Fast build tool with hot module replacement for React development.
- **psycpg2-binary:** PostgreSQL database adapter for Python enabling Supabase connectivity.

6.6 Module Description

The system consists of six main modules: Data Upload, Student Sectioning, Timetable Generation, Authentication, Room Booking, and Frontend Interface.

6.6.1 Data Upload

This module handles the ingestion of institutional data from Excel files. The scripts in the server/scripts directory parse rooms, students, and subject-faculty data, validate entries, and persist them to the Supabase database through SQLAlchemy ORM operations.

6.6.2 Student Sectioning

This module uses the LangGraph-based scheduler to intelligently group students into branch-based sections. The AI agent (via langchain-openai) proposes section groupings based on room capacities, while deterministic fallback logic ensures valid assignments if the AI produces incorrect output.

6.6.3 Timetable Generation

This module implements the core scheduling logic using the CP-SAT constraint solver. It models all hard constraints as boolean variables and interval constraints, then invokes the solver to find a feasible or optimal assignment of sessions to time slots, rooms, and faculty.

6.6.4 Authentication

This module manages user registration, login, and role-based access control. Passwords are stored with salt-based hashing for security. Three roles are supported: administrator, faculty, and club.

6.6.5 Room Booking

This module enables clubs to request room bookings and administrators to approve or reject them. It maintains booking status through pending, approved, and rejected states with optional admin notes.

6.6.6 Frontend Interface

This module provides the React-based web interface with role-specific views. Administrators see data upload controls and timetable management tools. Faculty see personalized schedule views. Club users see booking request forms and status tracking.

Chapter 7

Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through a system. It shows how data is processed, stored, and transferred between different components. DFD helps in understanding the system structure and provides a clear visualization of data processing stages.

7.1 Level 0 DFD

Level 0 data flow diagram represents the entire Camp-Nou system as a single process. The Administrator provides institutional data (Excel files) as input and receives generated timetables as output. Faculty members request and receive their personalized schedules. Club users submit booking requests and receive approval status updates. The system interacts with the Supabase database for all persistence operations.

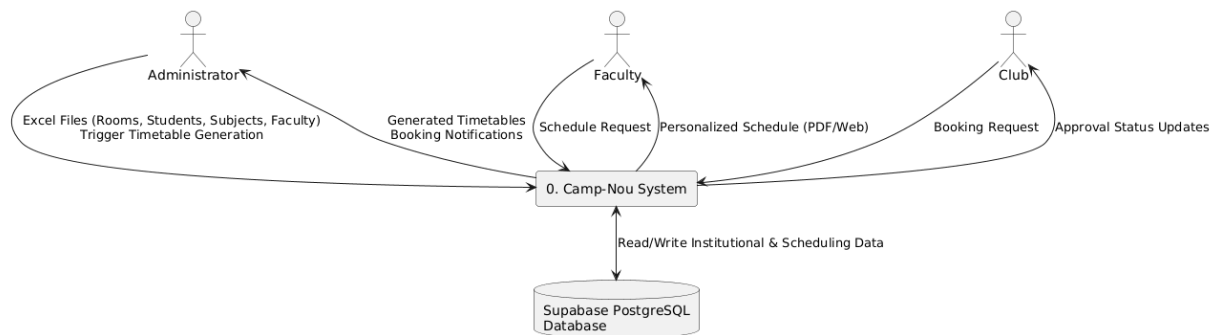


Figure 7.1: DFD Level 0

7.2 Level 1 DFD

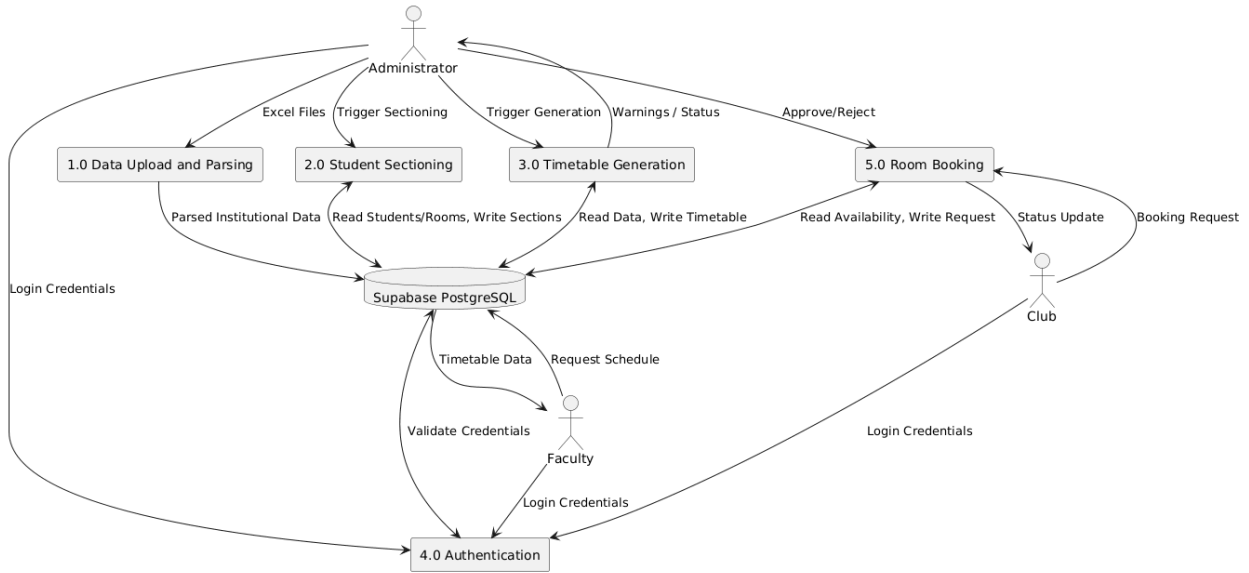


Figure 7.2: DFD Level 1

In Level 1 DFD, the system is divided into major sub-processes:

Data Upload and Parsing: This process accepts Excel file uploads from the administrator, parses them using Pandas, validates data integrity, and persists parsed records to the database.

Student Sectioning: This process reads student and room data from the database, uses AI to create balanced sections, and stores section-room assignments.

Timetable Generation: This process reads all institutional data, builds the constraint matrix, invokes the CP-SAT solver, validates the result, and persists the generated timetable.

Authentication: This process handles user login and registration, validates credentials against hashed passwords, and determines user roles for access control.

Room Booking: This process manages the lifecycle of room booking requests from submission through approval or rejection.

7.3 Level 2 DFD

A Level 2 DFD provides a detailed breakdown of the processes shown in Level 1.

7.3.1 Data Upload and Parsing

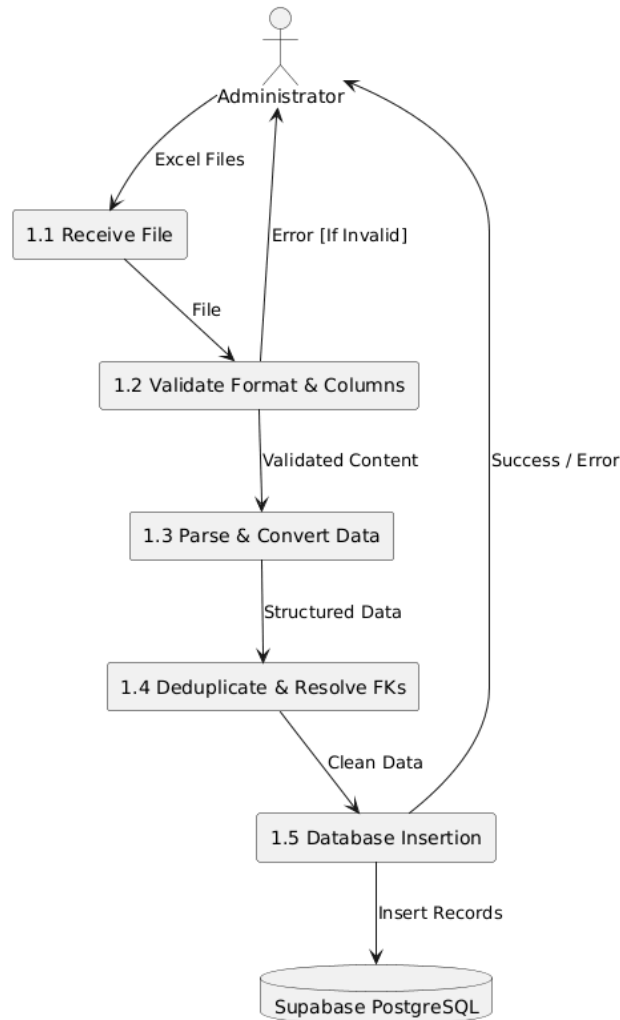


Figure 7.3: DFD Level 2

The data upload process includes file reception, format validation, column mapping, data type conversion, deduplication, and database insertion. Error handling ensures that invalid files produce meaningful error messages rather than silent failures.

7.3.2 Timetable Generation

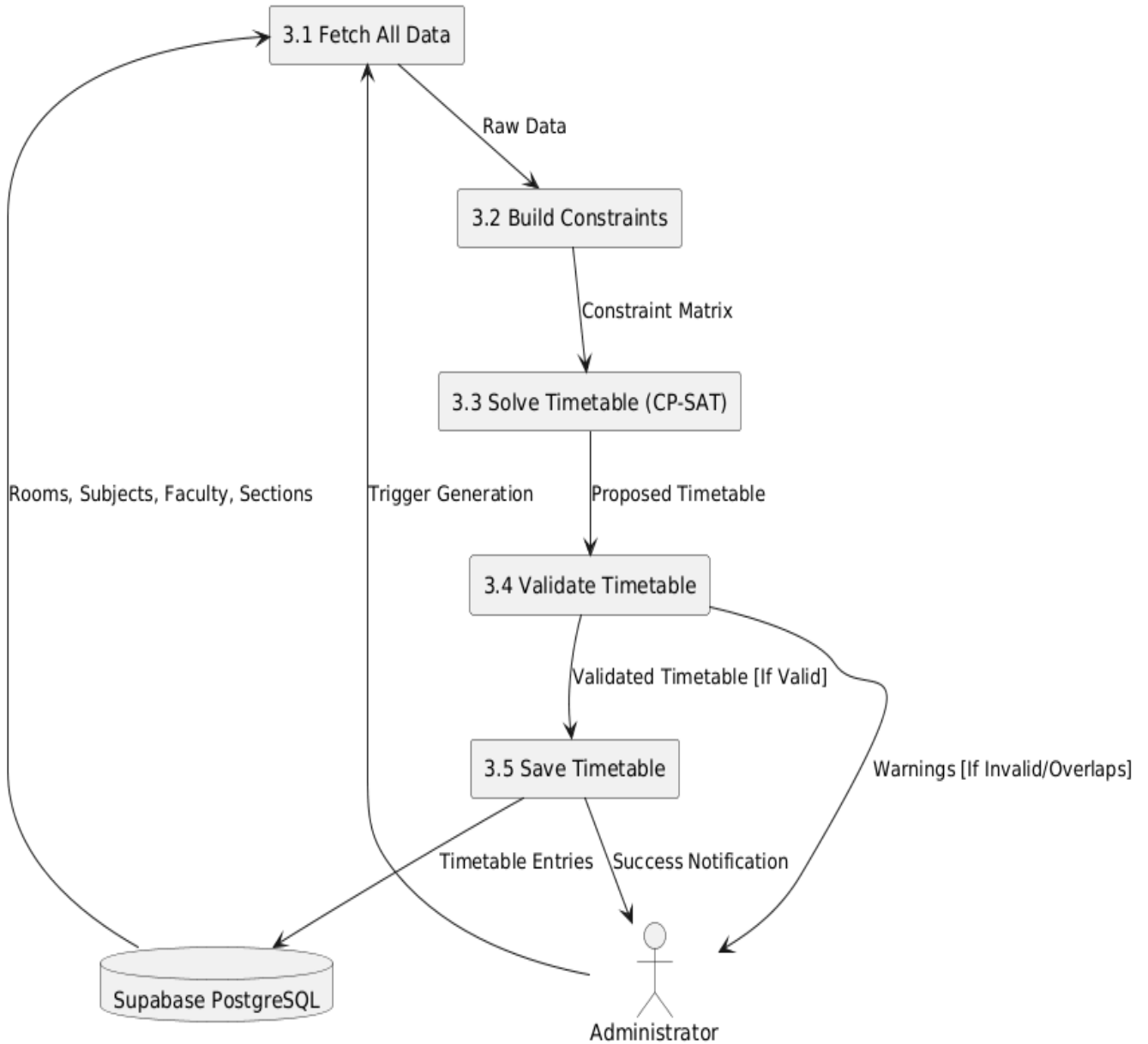


Figure 7.4: DFD Level 2

The timetable generation process is decomposed into five LangGraph nodes: `fetch_all_data` (database read), `build_constraints` (constraint matrix computation), `solve_timetable` (CP-SAT solver invocation), `validate_timetable` (double-booking verification), and `save_timetable` (database write). Conditional edges enable retry logic if validation fails.

7.3.3 Room Booking

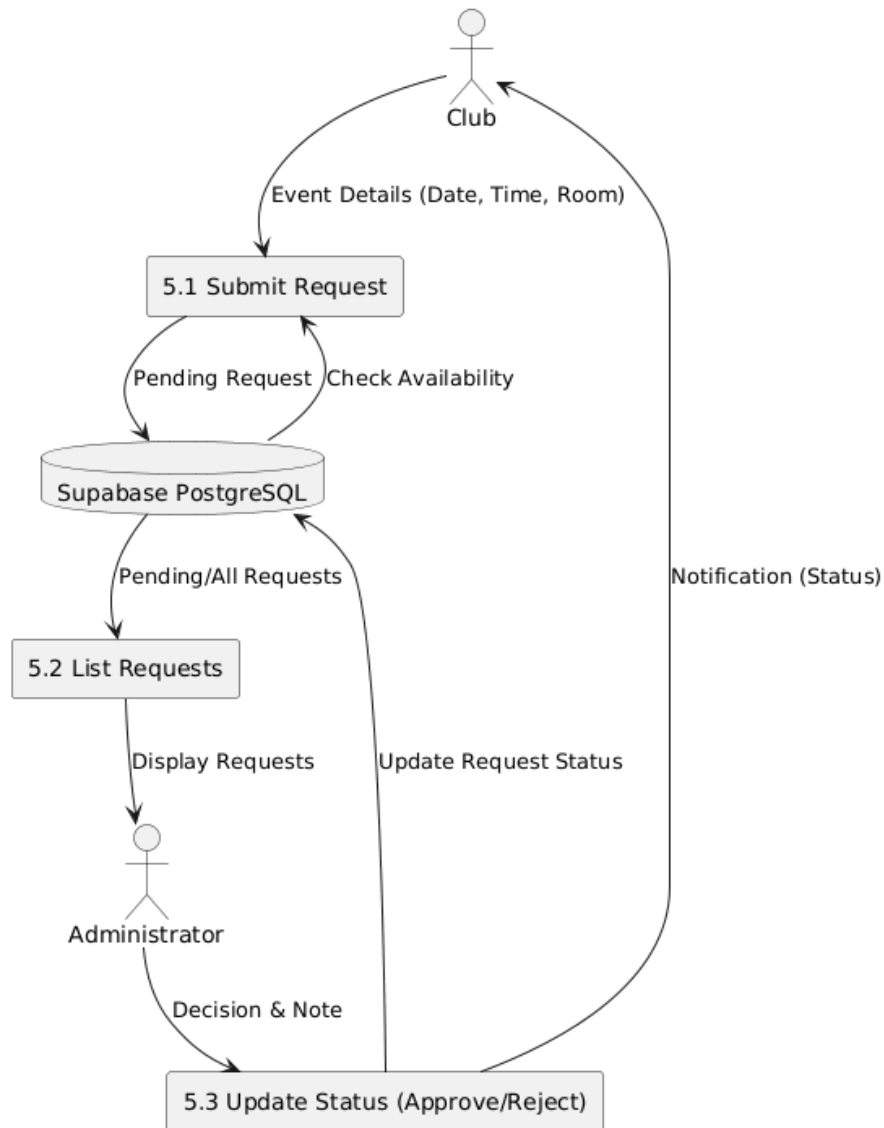


Figure 7.5: DFD Level 2

The room booking process includes request submission (by clubs), request listing (for administrators), status update (approve/reject with notes), and notification of the requesting club about the decision.

Chapter 8

Implementation

8.1 Algorithms Used

8.1.1 Student Sectioning Algorithm

The student sectioning process uses a two-tier approach combining AI-powered reasoning with deterministic fallback logic.

1. **Data Loading:** Student records and room configurations are loaded from the Supabase database via SQLAlchemy queries.
2. **AI Prompt Construction:** A structured prompt is constructed containing the full student list and room capacity constraints, requesting the LLM to produce branch-based section groupings in JSON format.
3. **LLM Invocation:** The prompt is sent to the AI model via langchain-openai's ChatOpenAI interface. The response is parsed from JSON, handling markdown code fences if present.
4. **Validation:** The AI response is validated to ensure every student appears in exactly one section and no section exceeds room capacity. If validation fails, the deterministic fallback is invoked.
5. **Deterministic Fallback:** Students are sorted by ID and sequentially assigned to sections based on maximum classroom capacity. This guarantees valid output regardless of AI performance.
6. **Room Allocation:** A second AI prompt assigns one classroom to each section with capacity constraints. Validation ensures no room is double-assigned.

8.1.2 Timetable Generation using CP-SAT

The timetable generation uses Google OR-Tools CP-SAT solver to produce mathematically guaranteed conflict-free schedules.

1. **Variable Definition:** For each teaching session, integer variables are created for day (0-5), slot (1-6), and room assignment. Lab sessions use interval variables spanning 3 consecutive slots.
2. **Hard Constraint Modeling:** Same-class sessions cannot overlap in time. Same-faculty sessions cannot overlap across any classes. Same-room sessions cannot overlap across any classes. Lab room assignments are restricted to rooms with `room_type="lab"`.
3. **Pairwise Constraint Enforcement:** For every pair of sessions, boolean variables model whether they share the same day, whether their time slots overlap, and whether they use the same room. Constraints enforce that overlapping conditions are mutually exclusive where required.
4. **Solver Execution:** The CP-SAT solver is invoked with a 60-second time limit. The solver uses constraint propagation and SAT-based search to find a feasible solution.
5. **Solution Extraction:** If a feasible or optimal solution is found, variable values are extracted and converted into the structured timetable format (class $\rightarrow$ day $\rightarrow$ slot $\rightarrow$ entry).
6. **Infeasibility Handling:** If the solver reports infeasibility, a descriptive warning is generated explaining that constraints cannot be simultaneously satisfied.
7. **Database Persistence:** Valid timetable entries are written to the Timetable table after clearing existing entries.

8.1.3 Authentication and Authorization

The authentication module implements secure credential management.

1. **Password Hashing:** User passwords are hashed with randomly generated salts before storage.
2. **Login Verification:** During login, the submitted password is hashed with the stored salt and compared against the stored hash.
3. **Role-Based Access:** API endpoints check the user's role to determine access permissions. Administrators have full access, faculty have read-only timetable access, and clubs have booking access.

8.2 Development Tools

8.2.1 Python Backend Stack

Python is used as the primary backend programming language.

1. **FastAPI:** Provides high-performance async API routing with automatic request validation and OpenAPI documentation generation.

2. **SQLAlchemy:** Provides ORM-based database access with transaction management and connection pooling.
3. **Uvicorn:** ASGI server for running the FastAPI application with hot reload during development.
4. **Pandas:** Used for parsing uploaded Excel files into structured DataFrames for database insertion.

8.2.2 AI and Constraint Programming

1. **LangChain:** Provides the ChatOpenAI interface for communicating with AI model endpoints.
2. **LangGraph:** Provides StateGraph abstraction for building multi-node agentic workflows with typed state management.
3. **OR-Tools CP-SAT:** Provides the constraint programming solver for guaranteed constraint satisfaction.

8.2.3 Frontend Development

1. **React:** Component-based UI library for building interactive role-specific dashboards.
2. **Vite:** Fast build tool with hot module replacement for rapid development iteration.
3. **CSS:** Custom styling for responsive layouts and role-specific visual theming.

Chapter 9

Unit Testing

Unit testing is performed to validate individual modules of the system including data upload, student sectioning, timetable generation, authentication, room booking, and the frontend interface. Each module is tested independently to ensure correct functionality.

Test Case and Test Results

Test Case Description	Test Steps	Expected Result	Actual Result	Status
Valid Rooms Excel Upload	Upload valid rooms.xlsx file with correct columns and data	System should parse file and insert room records into database	Rooms parsed and stored successfully	PASS
Valid Students Excel Upload	Upload valid students.xlsx file with name and branch columns	System should parse and store student records	Students stored correctly in database	PASS
Valid Subjects Faculty Upload	Upload subjects.faculty.xlsx with subject and mapping data	System should parse subjects, faculty, and mappings correctly	All records inserted with proper foreign keys	PASS
Invalid File Format Upload	Upload unsupported file (.txt/.jpg) as Excel data	System should reject invalid formats with error message	Error message returned properly	PASS
Empty Excel File	Upload Excel file with headers but no data rows	System should handle gracefully without crashing	Warning returned with zero records processed	PASS

Test Case Description	Test Steps	Expected Result	Actual Result	Status
Duplicate Data Upload	Upload same Excel file twice consecutively	System should handle duplicates without constraint violations	Existing records updated or skipped appropriately	PASS
Missing Required Columns	Upload Excel file with missing required columns	System should detect missing columns and return error	Clear error message identifying missing columns	PASS
Large Dataset Upload	Upload Excel file with 500+ student records	System should process large file within reasonable time	File processed successfully with all records stored	PASS

Table 9.1: Data Upload Module Testing

Test Case Description	Test Steps	Expected Result	Actual Result	Status
AI Student Sectioning	1. Upload student data 2. Upload room data 3. Trigger scheduler 4. Verify section groupings 5. Check capacity constraints	AI should group students into branch-based sections within room capacity limits	Students correctly grouped into CS-A, CS-B, EC-A sections within capacity	PASS
AI Validation Failure Fallback	1. Simulate AI returning invalid JSON 2. Trigger scheduler 3. Verify fallback activation 4. Check section assignments 5. Validate completeness	System should fall back to deterministic sorted-by-ID algorithm	Fallback triggered successfully with all students assigned	PASS
Room Allocation via AI	1. Complete student sectioning 2. Trigger room allocation 3. Verify room assignments 4. Check no duplicate rooms 5. Validate capacity match	AI should assign one classroom to each section with adequate capacity	Rooms allocated correctly without duplicates	PASS

Test Case Description	Test Steps	Expected Result	Actual Result	Status
CP-SAT Timetable Generation	1. Upload all institutional data 2. Trigger timetable generation 3. Verify solver execution 4. Check constraint satisfaction 5. Validate completeness	Solver should produce conflict-free timetable within time limit	Timetable generated with all sessions scheduled and no conflicts	PASS
Faculty Double-Booking Check	1. Generate timetable 2. Run validation node 3. Check faculty occupancy map 4. Verify no overlaps 5. Confirm validation passes	No faculty should be assigned to two classes in same day-slot	Validation passed with zero faculty conflicts	PASS
Room Double-Booking Check	1. Generate timetable 2. Run validation node 3. Check room occupancy map 4. Verify no overlaps 5. Confirm validation passes	No room should host two classes in same day-slot	Validation passed with zero room conflicts	PASS

Test Case Description	Test Steps	Expected Result	Actual Result	Status
Lab Consecutive Slots	1. Generate timetable with lab subjects 2. Verify lab entries 3. Check slot consecutiveness 4. Validate lab room type 5. Confirm 3-slot blocks	Lab sessions should occupy exactly 3 consecutive slots on same day	All lab sessions correctly span 3 consecutive slots	PASS

Table 9.2: Timetable Generation Testing

Test Case Description	Test Steps	Expected Result	Actual Result	Status
Admin Login	1. Navigate to login page 2. Enter admin credentials 3. Submit login form 4. Verify dashboard access 5. Check admin features visible	System should authenticate admin and show admin dashboard	Admin logged in successfully with full feature access	PASS
Faculty Login	1. Navigate to login page 2. Enter faculty credentials 3. Submit login form 4. Verify timetable view 5. Check limited access	System should authenticate faculty and show faculty dashboard	Faculty logged in with timetable view and PDF download access	PASS
Club Login	1. Navigate to login page 2. Enter club credentials 3. Submit login form 4. Verify booking interface 5. Check role restrictions	System should authenticate club and show booking interface	Club logged in with booking request and status tracking access	PASS

Invalid Credentials	1. Navigate to login page 2. Enter wrong email/password 3. Submit login form 4. Check error response 5. Verify no access granted	System should reject invalid credentials with error message	Login rejected with appropriate error message displayed	PASS
Role-Based Access Control	1. Login as faculty user 2. Attempt admin-only API call 3. Check response status 4. Verify access denied 5. Confirm security enforcement	System should block unauthorized role from admin operations	403 Forbidden returned for unauthorized access attempts	PASS
Password Hashing Security	1. Create new user account 2. Check database record 3. Verify password is hashed 4. Confirm salt is stored 5. Validate no plaintext	Passwords should be stored with salt-based hashing, never plaintext	Password hash and salt stored correctly in database	PASS

Session Persistence	1. Login successfully 2. Navigate between pages 3. Refresh browser 4. Check session state 5. Verify continued access	User session should persist across page navigations and refreshes	Session maintained correctly throughout user interactions	PASS
---------------------	--	---	---	------

Table 9.3: Authentication and Authorization Testing

Test Case	Test Steps	Expected Result	Actual Result	Status
Submit Booking Request	1. Login as club user 2. Navigate to booking page 3. Fill event details and room 4. Submit booking request 5. Verify request created	System should create pending booking request with event details	Booking request created successfully with pending status	PASS
Admin Approve Booking	1. Login as administrator 2. View pending bookings 3. Select booking to approve 4. Click approve button 5. Verify status change	Booking status should change from pending to approved	Booking approved successfully with status updated	PASS

Test Case	Test Steps	Expected Result	Actual Result	Status
Admin Reject Booking	1. Login as administrator 2. View pending bookings 3. Select booking to reject 4. Add rejection note 5. Verify status change	Booking status should change to rejected with admin note	Booking rejected with admin note stored correctly	PASS
Duplicate Time Booking	1. Create approved booking for room 2. Submit another request for same room and time 3. Check system response 4. Verify conflict detection 5. Validate appropriate handling	System should handle or warn about conflicting bookings	Conflict detected and handled appropriately	PASS
Booking Status Tracking	1. Login as club user 2. Submit booking request 3. Check booking status 4. Admin approves/rejects 5. Verify status updates visible	Club user should see updated booking status	Status updates reflected correctly in club dashboard	PASS

Test Case	Test Steps	Expected Result	Actual Result	Status
Default Room Seeding	1. Start application fresh 2. Check database for default rooms 3. Verify room configuration 4. Validate capacity values 5. Confirm room types set	System should seed default rooms on startup	Default rooms seeded correctly with proper configurations	PASS

Table 9.4: Room Booking Module Testing

Chapter 10

Graphical User Interface

10.1 GUI Overview

10.1.1 Description of the User Interface

The Camp-Nou application provides an intuitive and role-specific web interface designed to simplify the complex process of academic timetable management. The interface enables administrators to upload institutional data, generate timetables, and manage room bookings. Faculty members can view their personalized schedules and download PDF exports. Club representatives can submit and track room booking requests.

The system ensures ease of navigation by providing clearly defined role-based views. Each role sees only the features relevant to their responsibilities, reducing complexity and preventing unauthorized operations.

10.1.2 Objectives and Actions

The primary objective of the GUI is to provide a seamless interaction between different institutional stakeholders and the scheduling system. The application presents a unified platform where administrative data management, AI-powered schedule generation, and day-to-day room booking operations coexist within a single interface.

10.1.3 Interface Design Rules

- Provide role-specific navigation and feature visibility
- Ensure consistency across all screens and user flows
- Display meaningful feedback including success confirmations and error messages
- Maintain responsive layouts across desktop and tablet screen sizes
- Provide clear visual distinction between different timetable states (scheduled, suspended)

10.2 Main GUI Components

10.2.1 Components

1. **Login Interface:** Provides email and password authentication with role-based redirection.
2. **Admin Dashboard:** Central hub for data upload, timetable generation, and booking management.
3. **Data Upload Module:** Drag-and-drop or file selection interface for Excel file uploads.
4. **Timetable Grid View:** Tabular display of generated timetables organized by class, day, and slot.
5. **Faculty Schedule View:** Personalized timetable filtered by the logged-in faculty member.
6. **Room Booking Interface:** Form-based booking request submission with status tracking.
7. **Booking Management:** Admin view for reviewing and acting on pending booking requests.

10.3 Screen Images

10.3.1 Login Interface

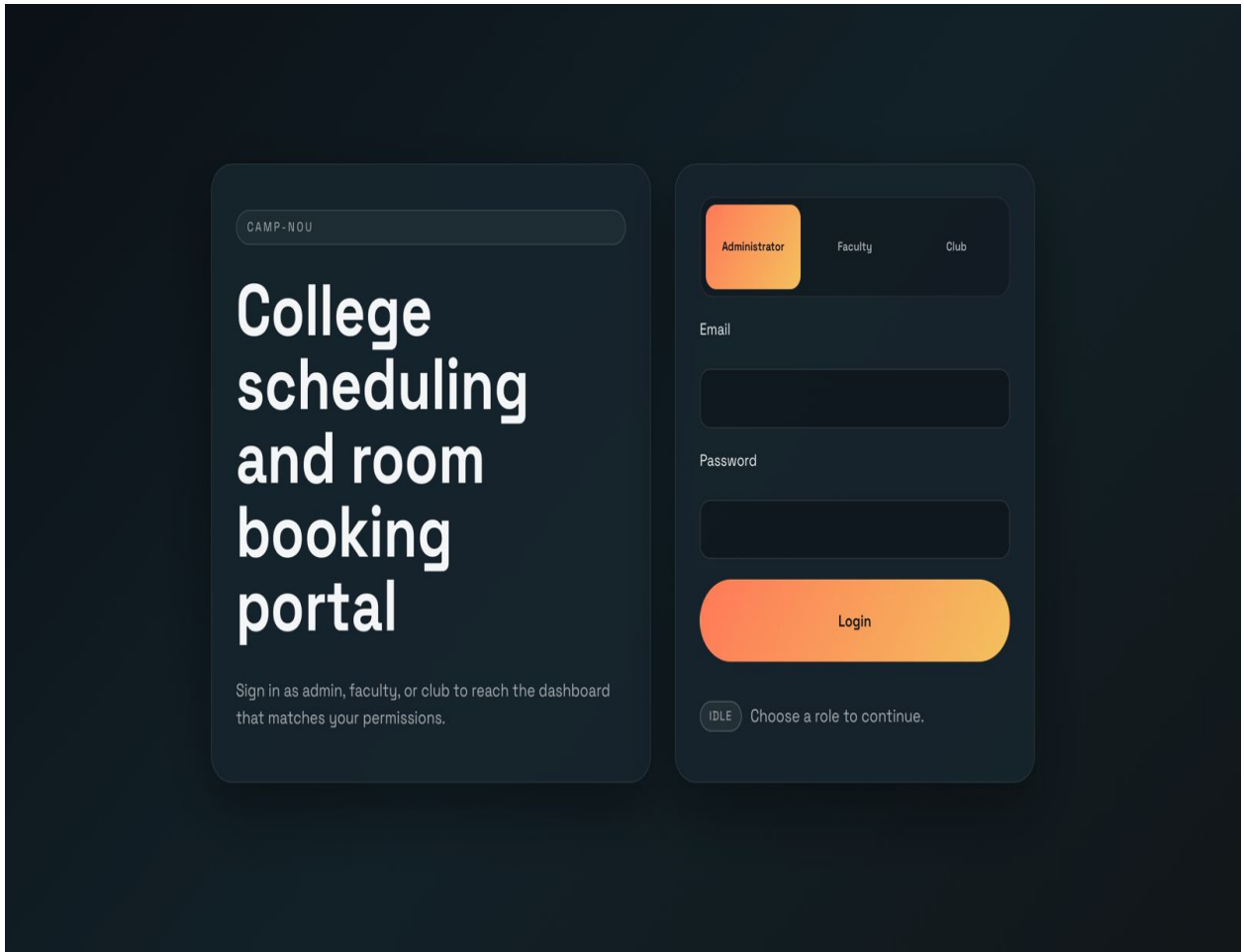


Figure 10.1: Login Interface

10.3.2 Admin Dashboard

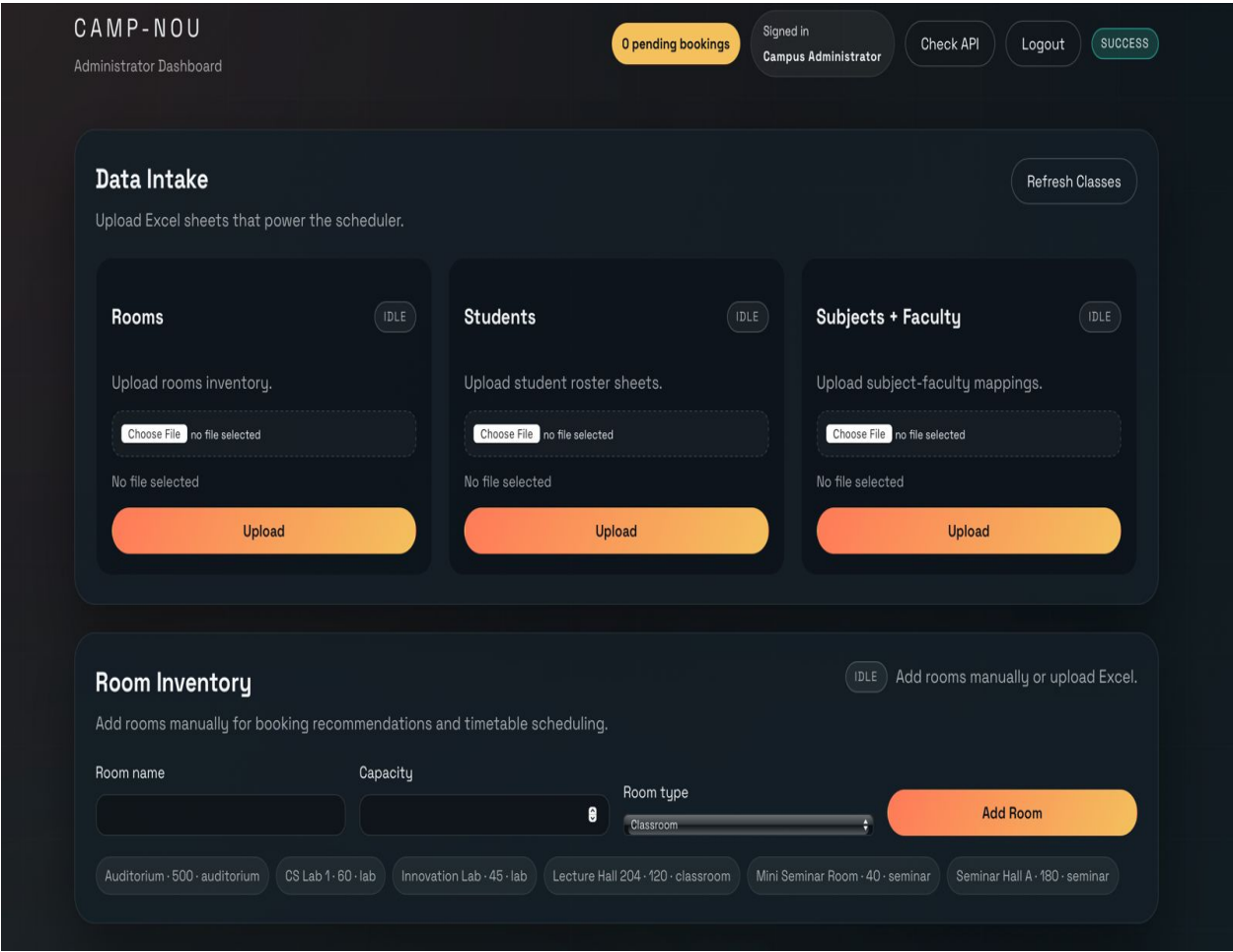


Figure 10.2: Admin Interface

10.3.3 Data Upload Module

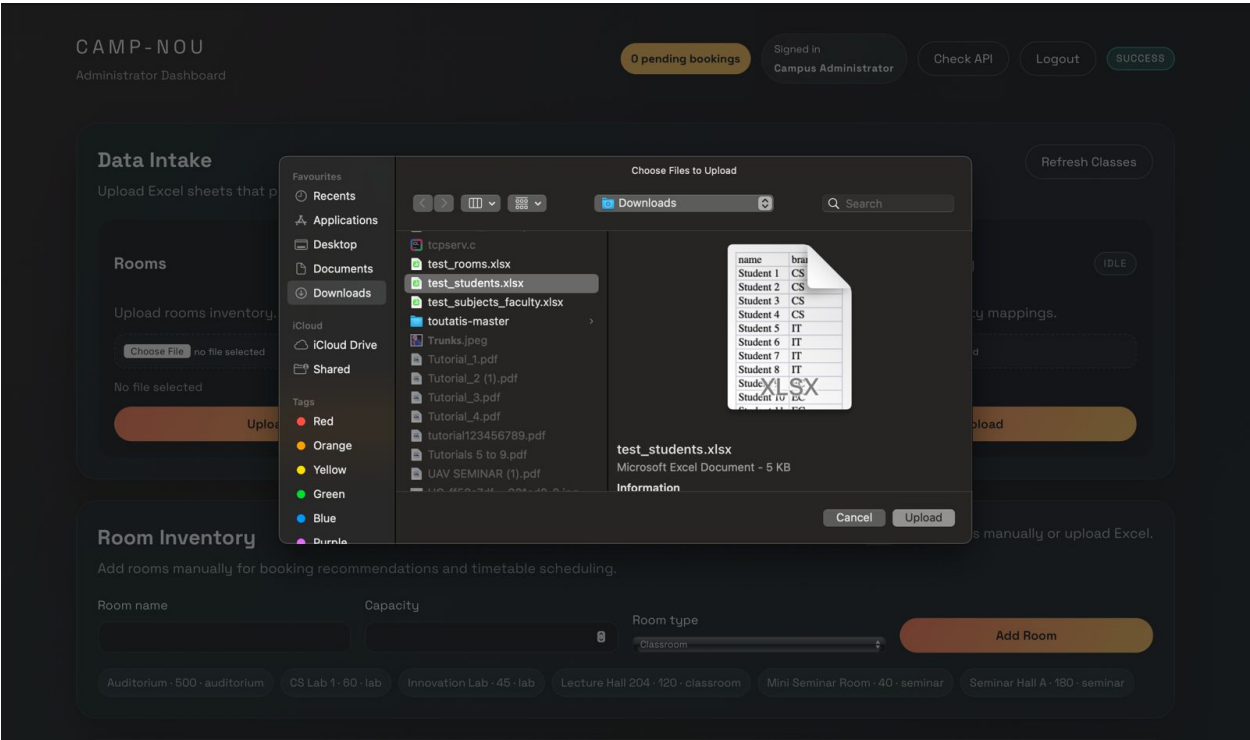


Figure 10.3: Data Upload Module

10.3.4 Timetable Grid View

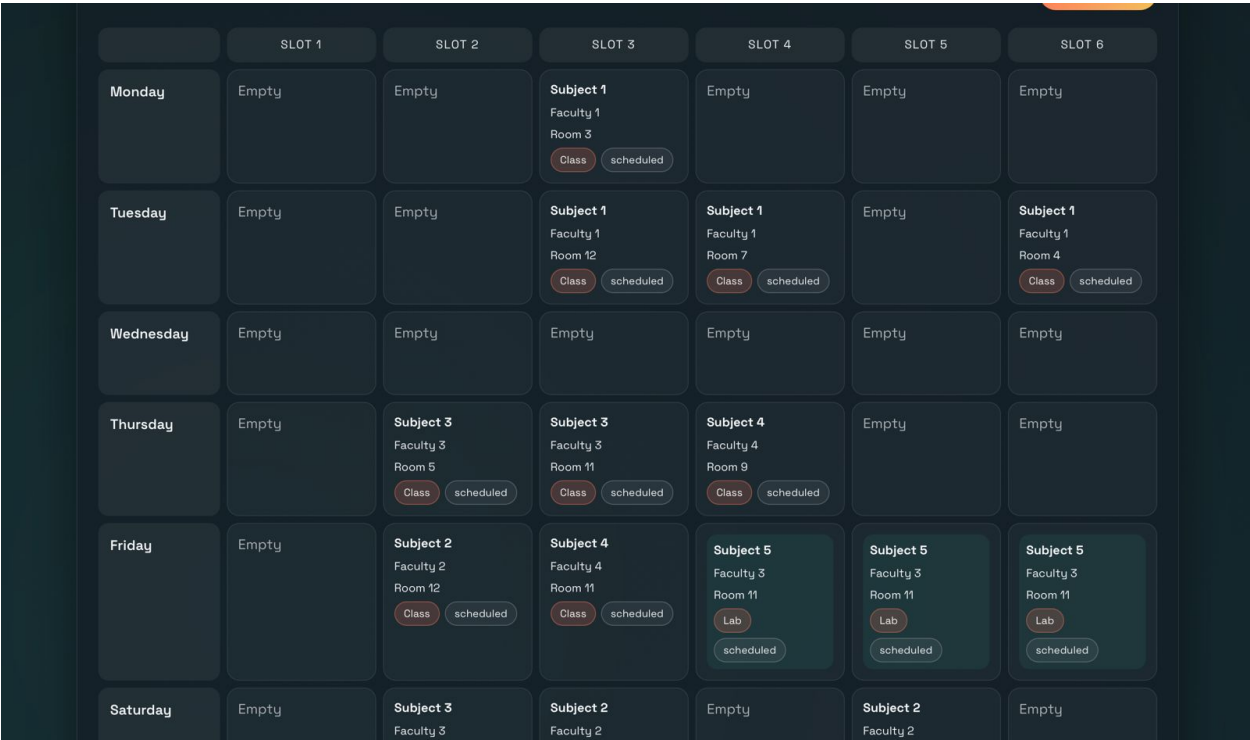


Figure 10.4: Timetable Grid View

10.3.5 Faculty Schedule View

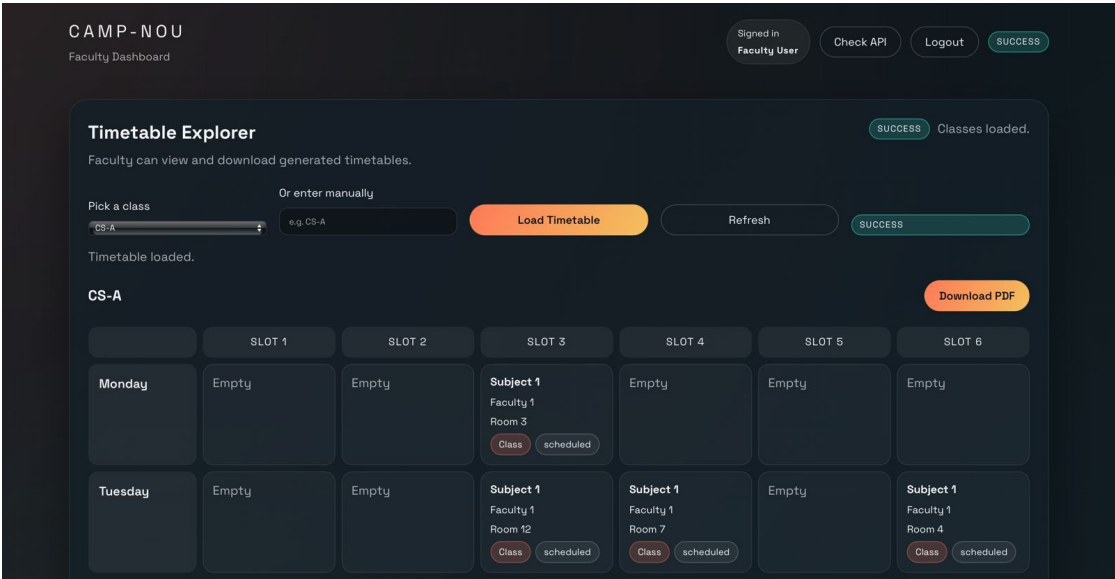


Figure 10.5: Faculty Schedule View

10.3.6 Room Booking Interface

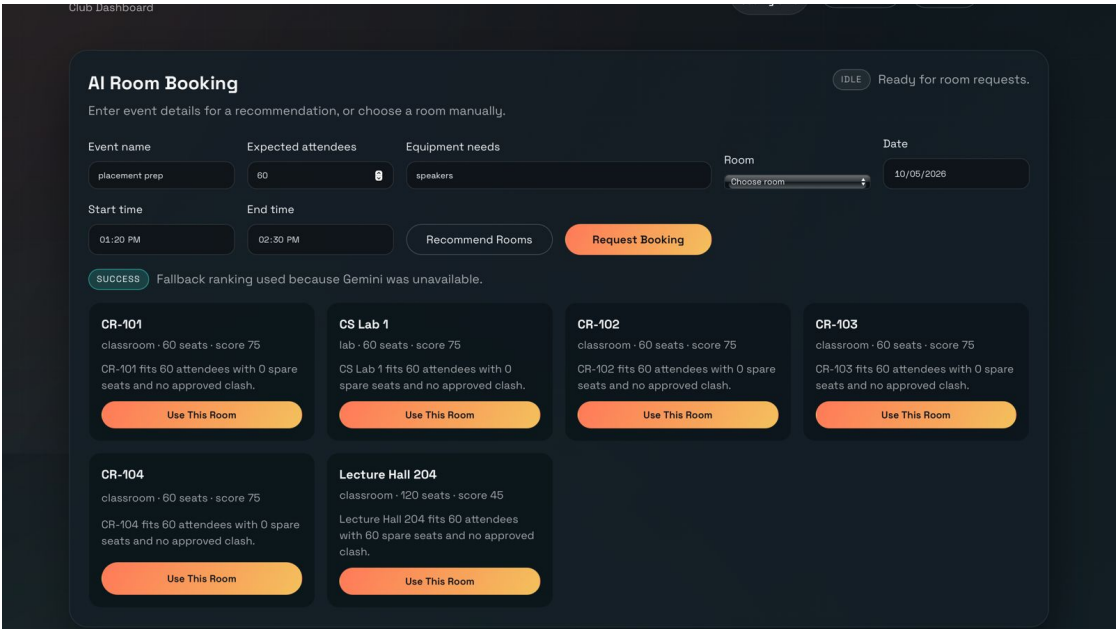


Figure 10.6: Room Booking Interface

10.3.7 Booking Management

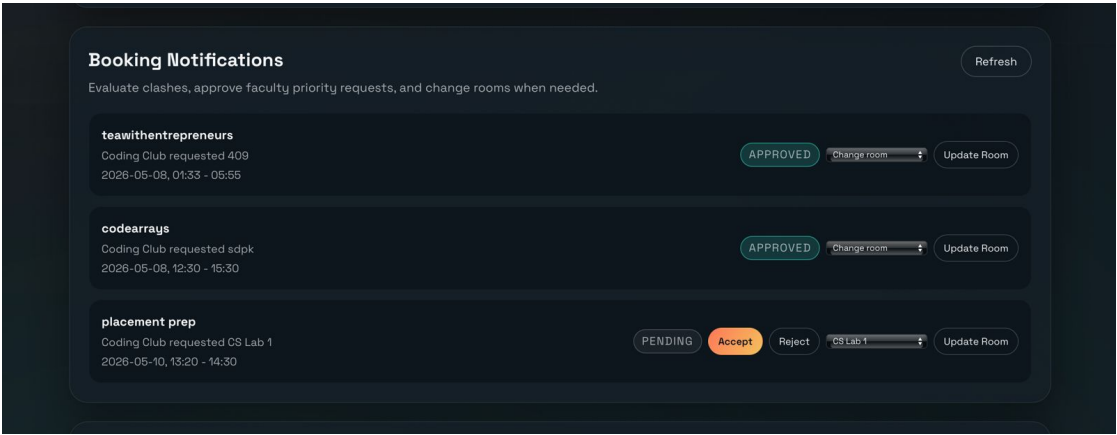


Figure 10.7: Booking Management

Chapter 11

Results

The Camp-Nou system was evaluated using representative institutional datasets to assess its performance in terms of scheduling correctness, generation speed, AI reliability, and usability across different user roles.

11.1 Performance Evaluation

11.1.1 Scheduling Correctness

The CP-SAT constraint solver consistently generates timetables with zero hard constraint violations. Faculty double-bookings and room conflicts are mathematically prevented through the constraint model. Lab sessions correctly occupy three consecutive slots on the same day in all generated schedules.

11.1.2 Generation Speed

The complete timetable generation pipeline — from data fetching through constraint building, solving, validation, to database persistence — executes within 30-90 seconds for typical institutional configurations. The CP-SAT solver finds feasible solutions well within its 60-second time limit for datasets with up to 10 class sections and 20+ subjects.

11.1.3 AI Reliability

The LLM-powered student sectioning module successfully produces valid branch-based groupings in the majority of invocations. When the AI produces invalid output (malformed JSON, student omissions, or capacity violations), the deterministic fallback mechanism activates seamlessly, ensuring system reliability. Warning logs track all fallback activations for monitoring.

11.2 Multi-Role Access Results

The role-based access control system correctly restricts features based on user roles:

- Administrators successfully upload data, generate timetables, and manage bookings

- Faculty members view only their personalized schedules with PDF download capability
- Club users submit booking requests and track approval status
- Cross-role access attempts are blocked with appropriate HTTP 403 responses

11.3 Room Booking Workflow

The Room Booking system operates correctly through the complete request lifecycle. Club submissions are recorded with pending status, administrators can approve or reject with notes, and status changes are reflected in the club user's dashboard.

11.4 Limitations

Despite its effectiveness, the system has certain limitations:

- CP-SAT solver may report infeasibility for severely over-constrained scenarios without detailed explanation of which constraints conflict
- AI student sectioning quality depends on the LLM model's capability and prompt engineering
- Real-time timetable modification during the academic term is not yet supported
- The system does not currently support multi-campus or distributed scheduling scenarios

11.5 Summary

Overall, the Camp-Nou system successfully demonstrates the application of hybrid AI and constraint programming approaches to academic timetable management. The combination of LLM-powered reasoning for flexible tasks with formal constraint solving for guaranteed correctness results in a reliable and adaptable scheduling platform.

Chapter 12

Conclusion

This project presents the design and implementation of Camp-Nou, an AI-powered college timetable orchestration system that combines Large Language Model reasoning with formal constraint programming to address the complex challenge of academic scheduling.

The system successfully demonstrates a hybrid architecture where an LLM handles flexible, creative tasks (student sectioning and room allocation) while the Google OR-Tools CP-SAT solver handles the mathematically rigorous timetable generation. LangGraph’s StateGraph abstraction provides structured workflow orchestration, ensuring that AI outputs are validated by deterministic code before database persistence.

The FastAPI backend provides a robust integration layer connecting the React frontend, Supabase PostgreSQL database, and AI/constraint programming engines. The multi-role access system enables end-to-end institutional workflow — from bulk data upload and AI-driven scheduling to individual schedule viewing and dynamic club room bookings.

The deterministic fallback mechanisms ensure that system reliability is never compromised by AI uncertainty. When the LLM produces invalid outputs, the system gracefully degrades to proven algorithmic approaches while logging warnings for monitoring.

The results demonstrate that this hybrid approach produces conflict-free timetables within acceptable time frames while maintaining the flexibility to adapt to new constraints without algorithmic rewrites. The platform serves as a foundation for intelligent college administration, reducing manual scheduling effort and enabling data-driven institutional management.

Chapter 13

Future Scope

The proposed system can be further enhanced in several directions. The CP-SAT solver can be extended with soft constraint optimization to improve the quality of generated timetables beyond mere feasibility. Multi-agent workflows utilizing separate AI models for different aspects of scheduling (section grouping, room assignment, conflict resolution) can improve overall system intelligence. Real-time websocket-based notifications can enable dynamic timetable updates and live booking status tracking. Integration with calendar systems (Google Calendar, Microsoft Outlook) can provide seamless schedule synchronization for faculty and students. Mobile application development can extend access to smartphones and tablets for on-the-go schedule checking. Support for multi-campus scheduling can address the needs of larger educational institutions. Advanced analytics dashboards can provide insights into room utilization rates, faculty workload distribution, and scheduling efficiency metrics.

References

- [1] J. Kingston, "Automated University Timetabling Using Constraint Logic Programming," *Journal of Educational Operational Research*, 2018.
- [2] M. W. Carter and G. Laporte, "Genetic Algorithms for University Course Timetabling," *International Computing Institute Press*, 2020.
- [3] Z. Yang *et al.*, "Large Language Models for Combinatorial Optimization and Constraint Satisfaction," *Proc. NeurIPS*, 2023.
- [4] H. Chase *et al.*, "Agentic Workflows in Educational Administration using LangChain," *AI Integrations Journal*, 2024.
- [5] L. Perron and V. Furnon, "OR-Tools CP-SAT Solver for Scheduling Problems," *Google Research*, 2022.
- [6] S. Newman, "Multi-Role Web Applications for Educational Institution Management," *Journal of Web Engineering*, 2023.
- [7] S. Ramirez, "FastAPI and SQLAlchemy for High-Performance Backend Services," *Python Web Development Conference*, 2023.
- [8] A. Patel and R. Kumar, "Room Booking and Resource Management Systems for Universities," *International Journal of Smart Campus Technologies*, 2024.
- [9] D. Abramov *et al.*, "React-Based Single Page Applications for Dashboard Interfaces," *Frontend Engineering Review*, 2024.
- [10] T. Fischetti and M. Lodi, "Hybrid AI and Constraint Programming Approaches for Resource Allocation," *Proc. AAAI Conference on Artificial Intelligence*, 2025.
- [11] FastAPI Documentation, [Online]. Available: <https://fastapi.tiangolo.com/>
- [12] Google OR-Tools Documentation, [Online]. Available: <https://developers.google.com/optimization>
- [13] LangChain Documentation, [Online]. Available: <https://python.langchain.com/>
- [14] React Documentation, [Online]. Available: <https://react.dev/>
- [15] Supabase Documentation, [Online]. Available: <https://supabase.com/docs>